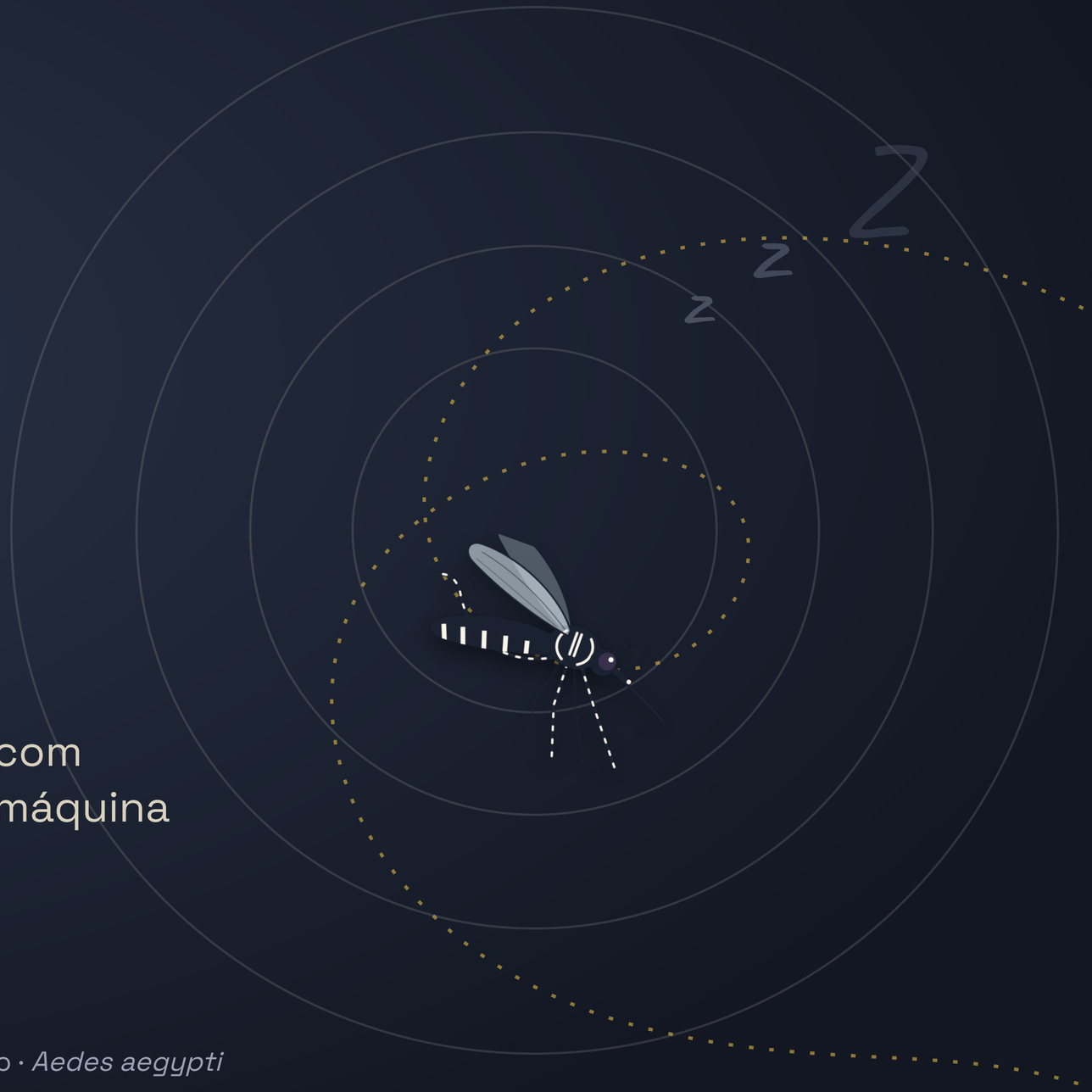
IFSC · Aprendizado de Máquina · 2026

Zzzz... é *Aedes*?



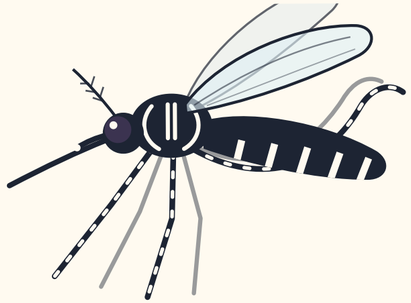
Ouvindo o batimento de asa dos mosquitos com **Machine Learning** — e investigando o que a máquina *realmente* escuta.

Luiz Eduardo do Nascimento Ciarallo

Projeto `mosquito-wingbeat` · detector ternário: ruído · outro mosquito · *Aedes aegypti*

o problema

Por que ouvir mosquitos?



Um vetor perigoso

O *Aedes aegypti* transmite **dengue, zika e chikungunya**. Saber **onde e quando** ele aparece orienta o combate.



Microfone barato

Uma armadilha com microfone pode monitorar **24 h por dia**, sem ninguém olhar lupa. O som vira dado.



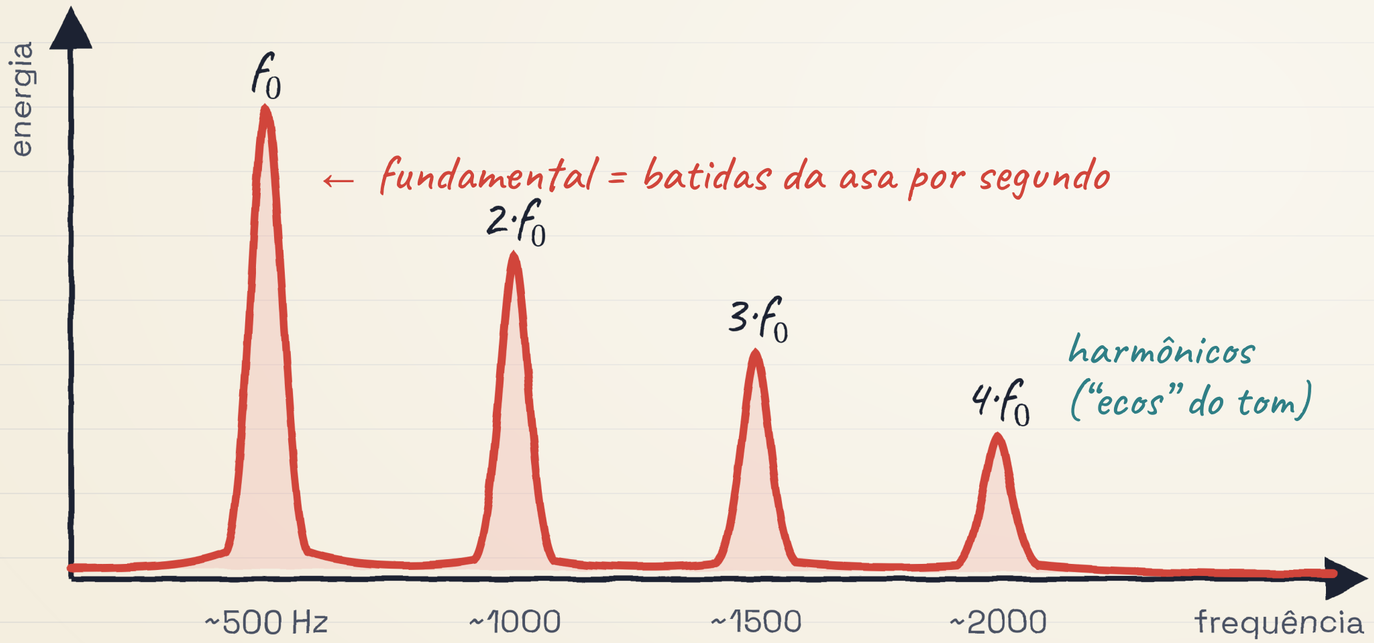
~500 batidas de asa por segundo

Cada espécie “canta”

O zumbido é o **batimento de asa**: centenas de Hz, com harmônicos. A ideia é reconhecer a espécie **pelo tom**.

a física do zumbido

O canto: um **tom** e seus **ecos**



como medimos f_0

```python
# candidatos de 250 a 1000 Hz
cand = np.arange(250, 1001)
# soma harmônica (média dos logs)
harm = np.exp(sum(np.log(amp(k*cand))
          for k in range(1, 4)) / 3)
f0 = cand[np.argmax(harm)]  # o "tom"
```

⚠ Mas os tons **se sobrepõem:** Aedes aparece de ~300 a ~900 Hz, outras espécies de 250 a 1000 Hz. Macho ≠ fêmea. Só o tom não basta.

os dados

Muitos arquivos, poucas histórias independentes

Dryad / Abuzz
Celulares, vários labs.
20 Aedes · **549** outros

HumBug
Campo, Tanzânia.
40 Aedes · **464** outros

ESC-50 + Pete
Ruídos genéricos.
542 arquivos sem mosquito

Externos
MosquitoSong + UFRGS
nunca usados no treino



agrupando por origem



vermelho = Dryad · azul = HumBug

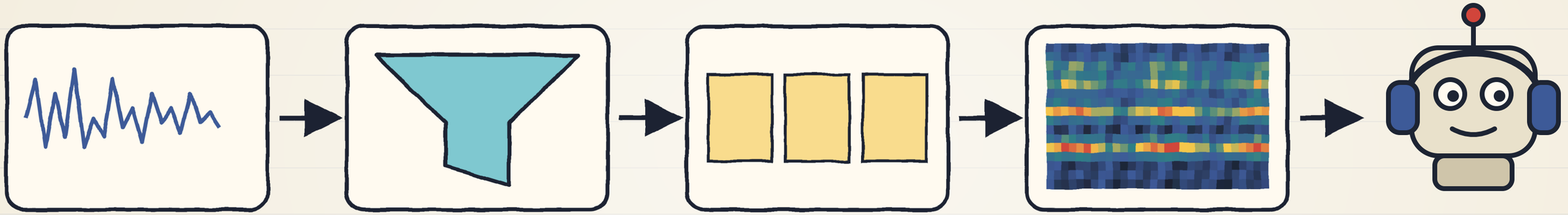
60 arquivos de Aedes no desenvolvimento...

...mas só **10 potes** independentes (6 Dryad + 4 HumBug)

cluster = laboratório+aparelho (Dryad) · local+aparelho+dia (HumBug) · fonte: `reanalysis/outputs/inventory_summary.csv`

como a máquina "ouve"

Do zumbido aos **números**



áudio bruto
.wav, .mp4, celular...

8 kHz · 100–3400 Hz
mesmo filtro p/ todos

janelas de 1 s
volume normalizado

log-mel 40 × 59
uma "foto" do som

modelo
ruído · outro · Aedes

o espectrograma, em 4 linhas

```
frames = sliding_window_view(w, N_FFT)[::HOP]   # 59 quadros
spec = np.abs(np.fft.rfft(frames * HANN)) ** 2   # potência
m = np.log(spec @ FB.T + 1e-6).T                 # 40 bandas mel
return m - m.mean()                              # tira o volume
```

*A escala **mel** imita o ouvido: mais detalhe nos graves, menos nos agudos. Além da "foto", guardamos **9 descritores físicos** do batimento (f_0, harmônicos, centroide...).*

fonte: `reanalysis/frontend.py · logmel()`

os métodos de ML

Quatro jeitos de escutar

1 Regressão logística

Uma **balança**: cada banda de frequência ganha um peso; soma tudo e decide.

```
sc = StandardScaler().fit(X)
m = LogisticRegression(C=param).fit(
        sc.transform(X), y, sample_weight=w)
```

2 Gradient Boosting (HistGB)

Centenas de **árvores de perguntas** "sim/não", cada uma corrigindo a anterior.

```
m = HistGradientBoostingClassifier(
        learning_rate=0.05, max_iter=300,
        max_leaf_nodes=param).fit(X, y)
```

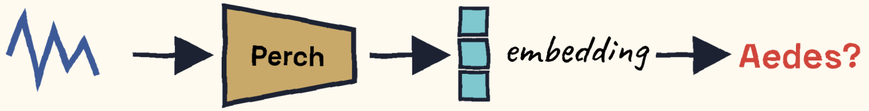
3 Rede convolucional (CNN)

Olha o espectrograma **como uma imagem** e aprende os próprios filtros.

```
block = lambda i, o: [nn.Conv2d(i, o, 3, padding=1),
    nn.BatchNorm2d(o), nn.ReLU(), nn.MaxPool2d(2)]
net = nn.Sequential(*block(1,16), *block(16,32), *block(32,64),
    nn.AdaptiveAvgPool2d(1), nn.Flatten(), nn.Linear(64, 3))
```

4 Perch 2.0 + cabeça leve

Um "ouvido" do Google **pré-treinado em sons de aves.** Transforma 5 s de áudio num vetor; uma logística decide em cima dele (v3/v4: congelado ou afinado na GPU).



fonte: `reanalysis/train_v2.py` · `fit_tabular()`, `make_cnn()` · `v3_embed.py`

a regra de ouro

Não deixar o modelo colar na prova

✗ dobras por arquivo



o mesmo celular aparece nos dois lados

✓ dobras por pote (cluster)



a prova é de um aparelho nunca visto

o "fiscal" da prova

```
sg = StratifiedGroupKFold(n_splits=5)
for tr, te in sg.split(X, y, groups=cluster):
    assert not (set(cluster[tr])
                & set(cluster[te])), \
        "cluster leakage"
```

Na versão antiga, **81 de 110 sessões** do Dryad caíam ao mesmo tempo no treino e na prova.

Agora tudo é **pré-registrado** e congelado por SHA-256 antes de treinar.

resultado 1

O brilho **enganoso**

mesmos aparelhos no treino e na prova.

90,03%

RETROSPECTIVO

de acurácia na cascata histórica · **47 de 60** grupos de Aedes detectados



...e a queda

Quando uma **fonte inteira** saiu do treino, a detecção de Aedes despencou de **75–100%** para **0–12%**.

Pior: o corpus Torres era **100% Aedes** e o ESC-50 **100% ruído**. Acertar a *fonte* já bastava para acertar o *rótulo*.

fonte: `reports/historico_ternario.md` · `reports/reanalise_v2.md §1`

resultado 2 · reanálise v2

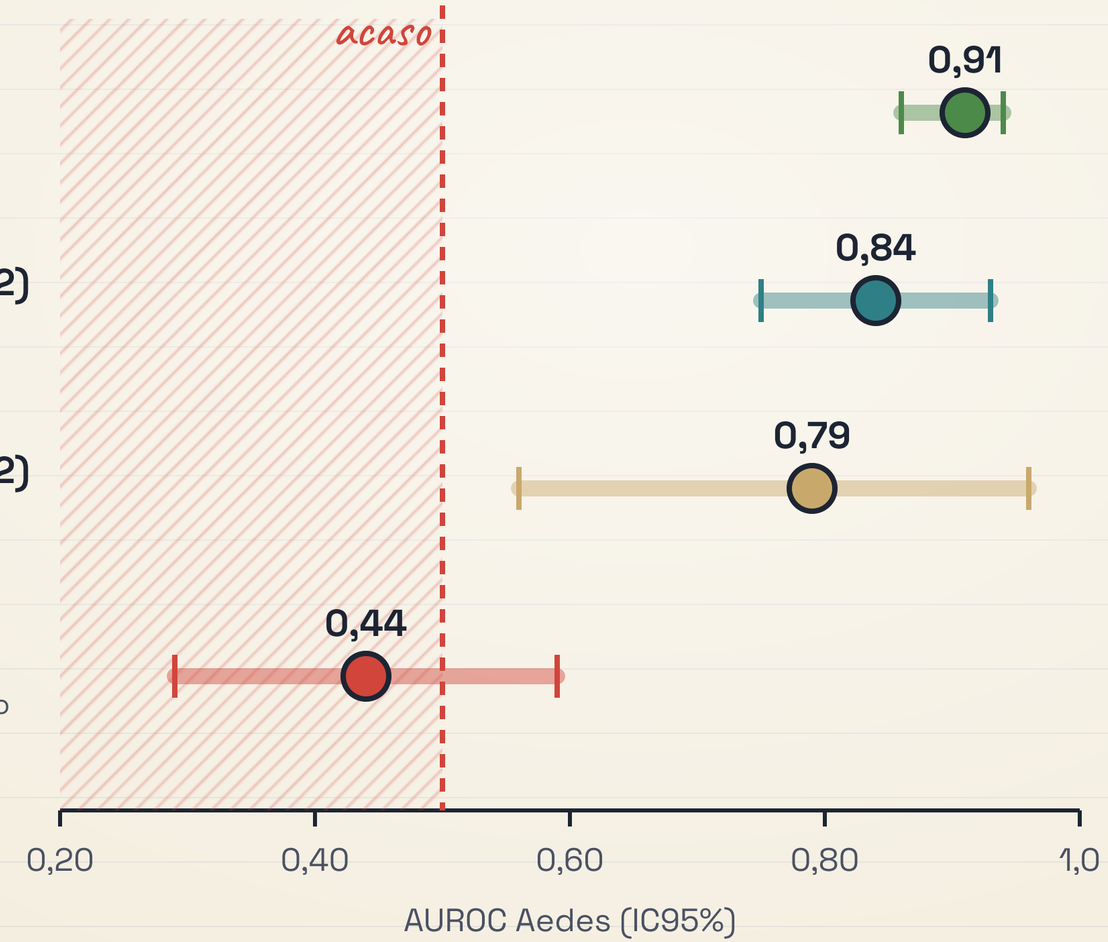
Quanto mais longe de casa, pior

Mesmas fontes (E1)
HistGB

Dryad → HumBug (E2)
CNN

HumBug → Dryad (E2)
HistGB · acurácia bal. ≤ 0,25

MosquitoSong (E3)
nunca visto · melhor modelo



AUROC em 10 segundos
Probabilidade de um Aedes receber nota maior que um não-Aedes. **0,5 = cara ou coroa**; 1,0 = perfeito.

UFRGS
CNN, HistGB e logística detectaram **0 de 10** Aedes.

*Mesmo "em casa", no limiar de decisão a **maioria dos alertas era falsa** (precisão 14–46%).*

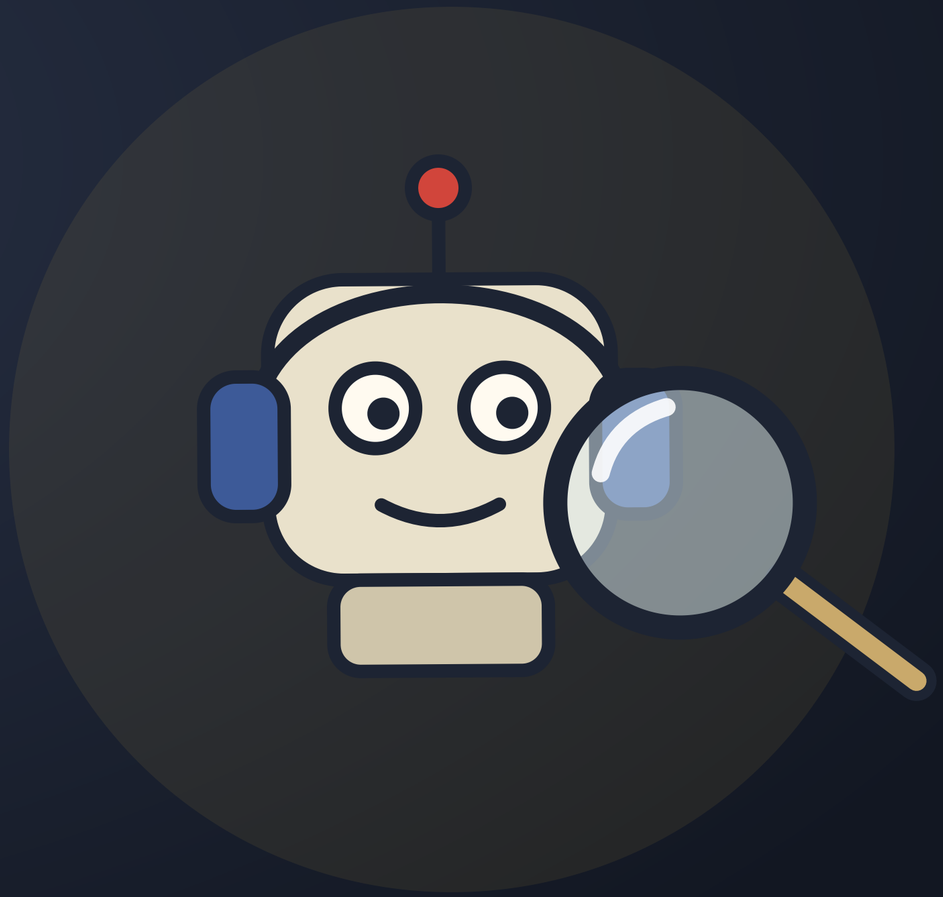
IC95% por bootstrap de cluster (2.000 réplicas) · fonte: `reports/reanalise_v2.md §5`

parte 2

O que a máquina está **ouvindo**?

Acurácia diz **quanto** o modelo acerta.
Interpretabilidade pergunta **por quê**.

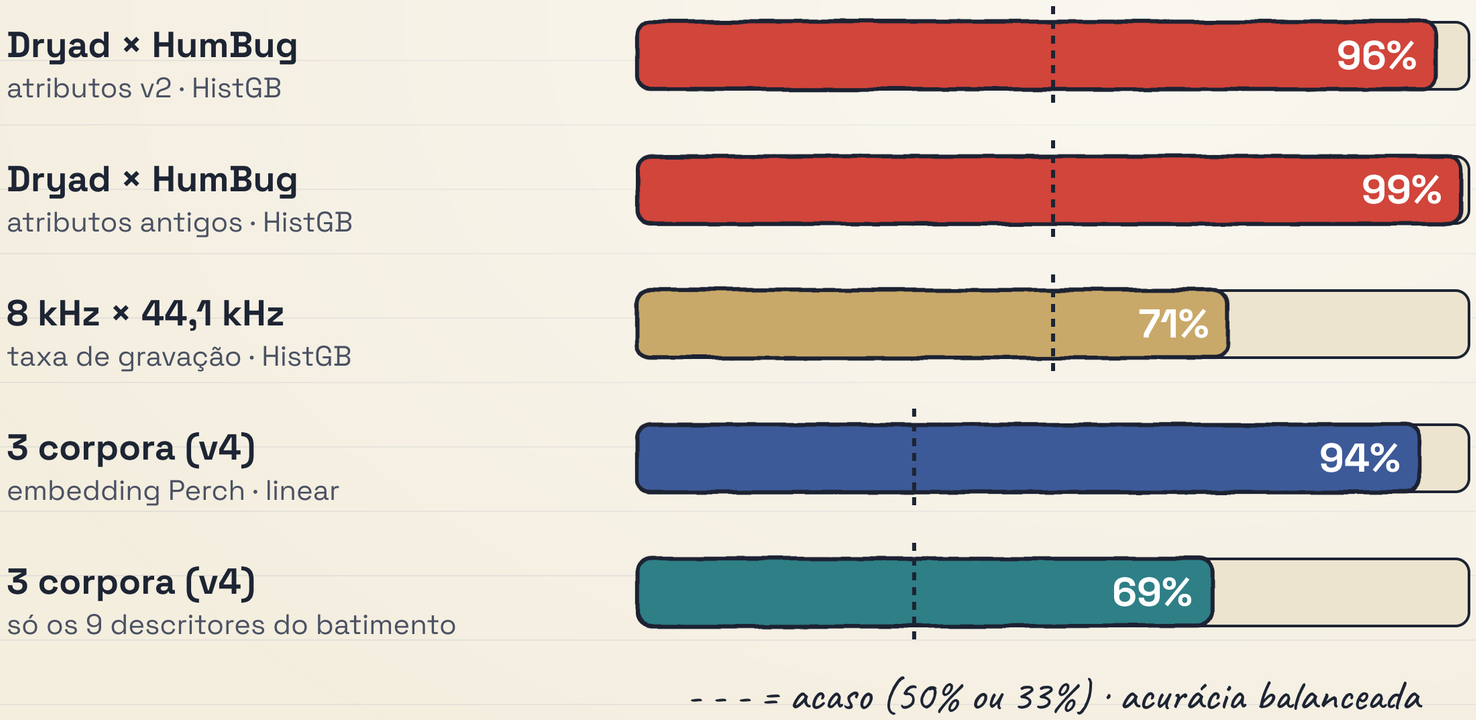
🔍 pista 1 · a origem
🎧 pista 2 · tampar bandas
📣 pista 3 · mosquitos falsos
📈 pista 4 · o espectro

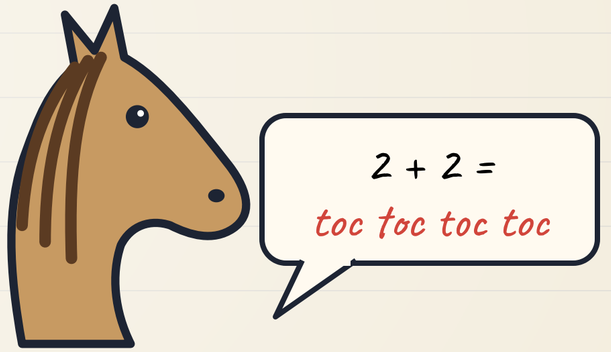


pista 1 · probe de origem

O microfone denuncia

Treinamos um classificador para adivinhar **de onde veio** a gravação, não a espécie.
Se ele acerta fácil, o detector também pode "colar".





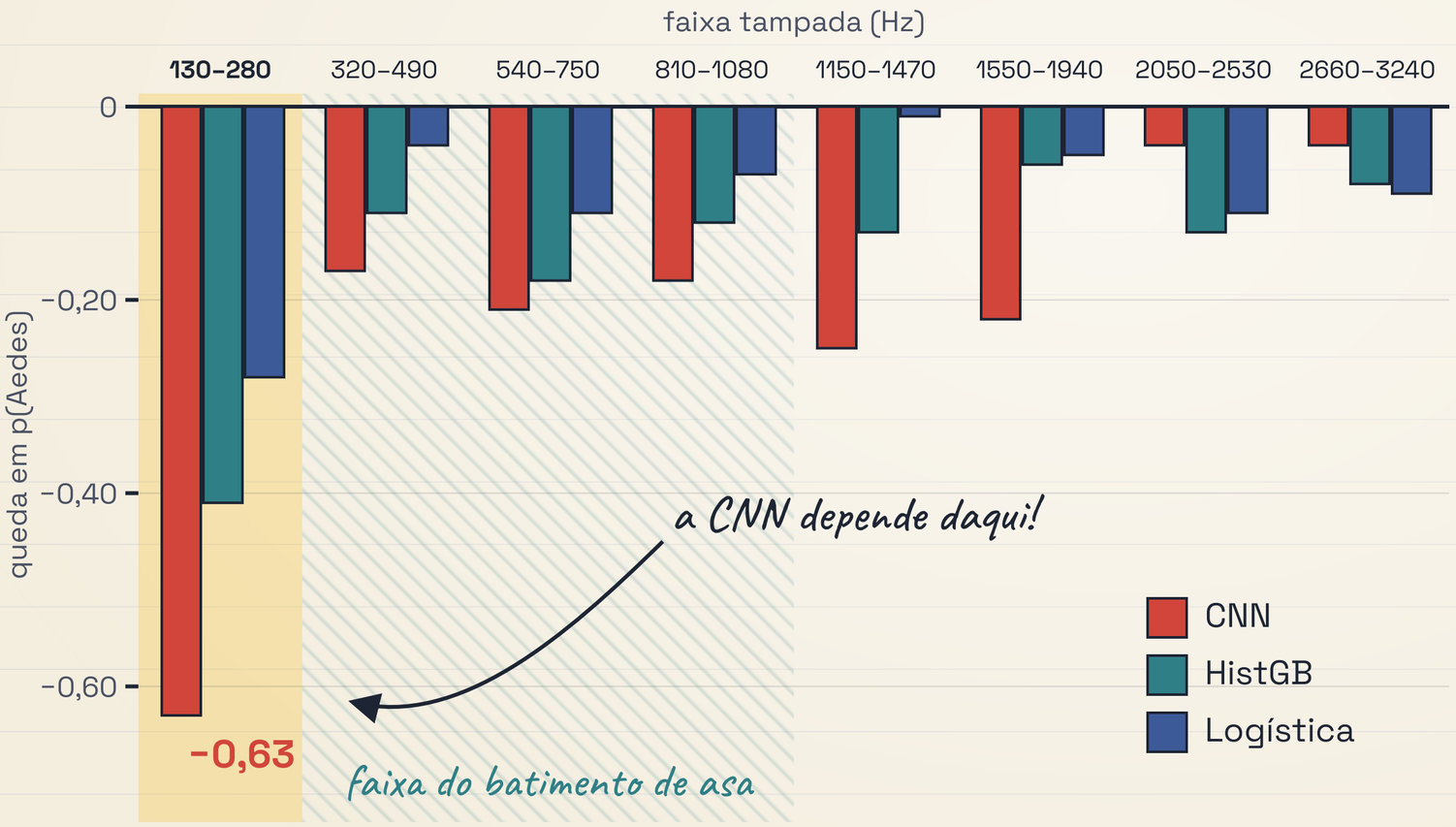
Efeito "Cavalo Esperto"

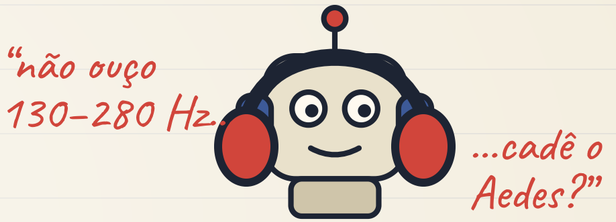
Hans, o cavalo, "fazia contas" lendo o rosto do dono. Nosso modelo pode "achar Aedes" lendo o **aparelho**. Parte do sinal é biológica (as fontes têm espécies diferentes), mas o alerta pré-registrado (> 80%) disparou.

fonte: `reanalysis/outputs/shortcut_probe.json` · `reports/resultados_v4.md`

pista 2 · oclusão de bandas

Tampando os ouvidos, uma faixa por vez





o experimento

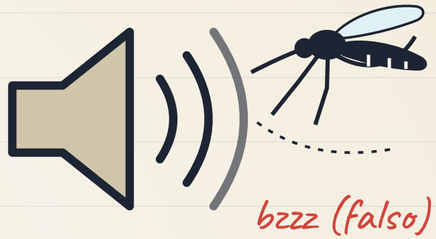
```
for g in range(8):          # 8 faixas
    bands = np.arange(5*g, 5*g + 5)
    M2 = M.copy(); M2[:, bands, :] = 0
    p = model(M2)           # de novo
    delta = p - base        # caiu?
```

*A maior queda fica **abaixo** do batimento de asa — onde mora a assinatura do aparelho/codec, não a do mosquito.*

pista 3 · teste de estresse

Mosquitos de mentira

Geramos sons que **parecem** zumbido mas não são: tons puros e harmônicos de 300–900 Hz, zumbido elétrico, ruído rosa e branco.



Logística

12/26

tons artificiais chamados de **“Aedes”** (7/13 puros, 5/13 harmônicos).

cai em qualquer apito

HistGB

8/13

tons harmônicos chamados de **“outro mosquito”**. Nenhum tom puro enganou.

harmônico = mosquito?

CNN

0/38

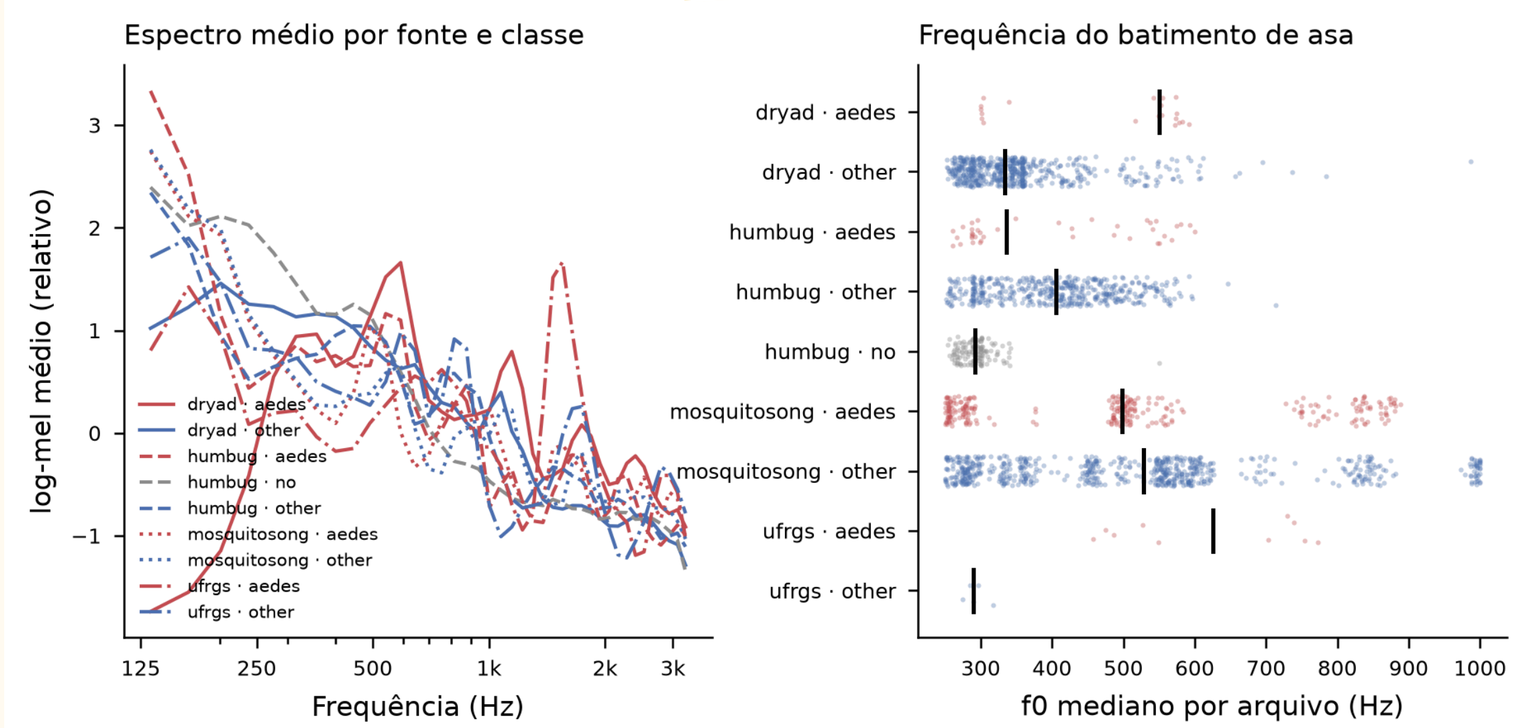
sons falsos aceitos: rejeitou tudo! Mas chamou **54%** dos Aedes **reais** do MosquitoSong de ruído.

desconfiada demais

Nenhum modelo é robusto e coerente ao mesmo tempo. fonte: `outputs/S4_summary.csv` · `reports/reanalise_v2.md §6`

pista 4 · olhando os dados

Cada fonte tem sua “voz”



Esquerda: as curvas se agrupam mais por **fonte** do que por espécie — o Dryad Aedes (vermelho contínuo) tem um formato próprio nos graves.

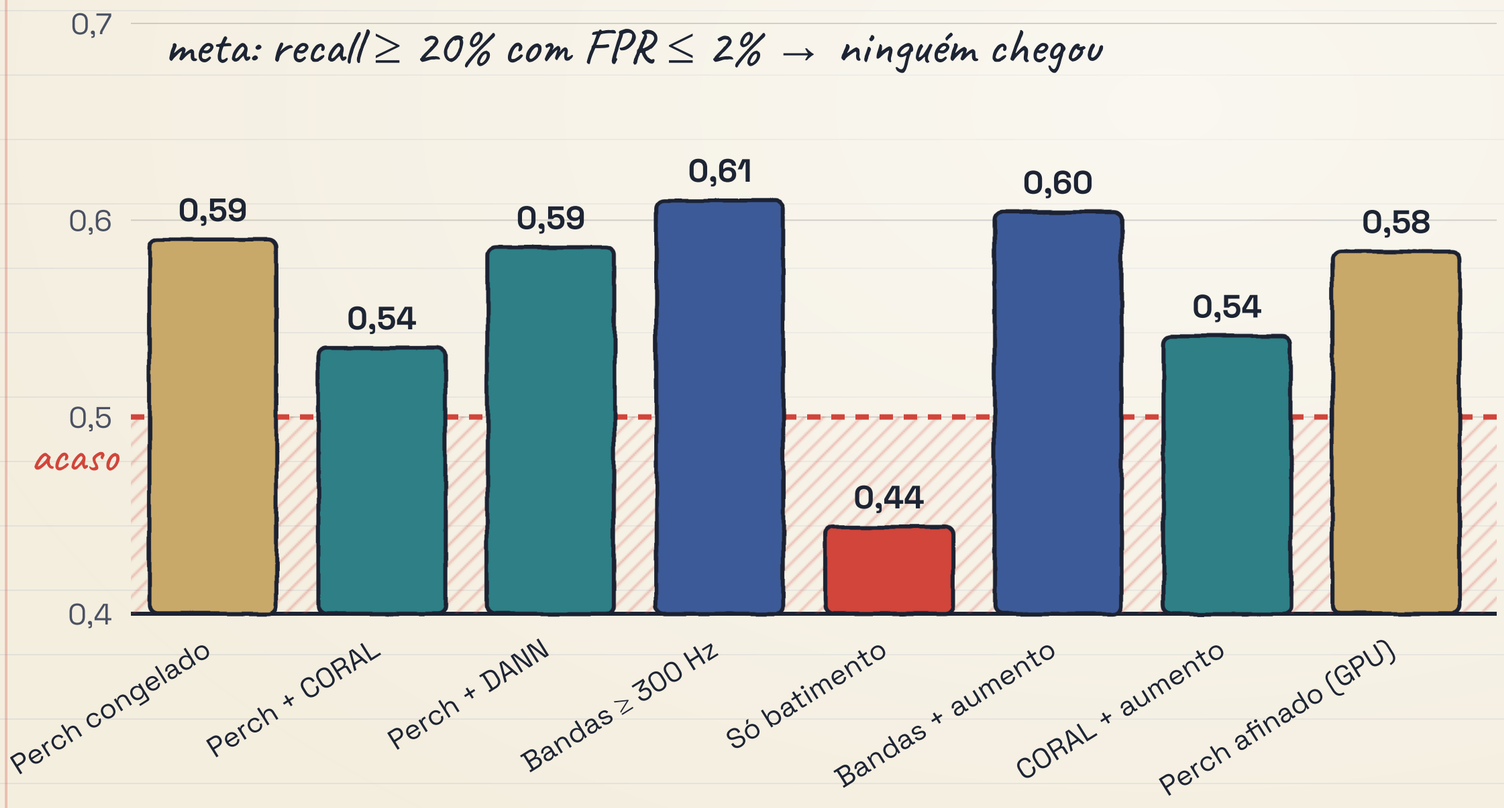
Direita: o f_0 de Aedes e de outras espécies se **mistura** (≈ 280–880 Hz).

*17 de 20 Aedes do Dryad foram gravados a **8 kHz**, contra ~31% das outras espécies. E todos os Aedes do HumBug vêm de **um único setup**.*

v3 e v4 · tentando consertar

8 remédios, nenhuma cura

AUROC médio deixando cada corpus de fora (Abuzz, HumBug, MosquitoSong)



DANN: o espelho do gradiente

```python
class GradReverse(autograd.Function):
    # ida: não muda nada
    def forward(ctx, x, lam): return x
    # volta: inverte o sinal → a rede
    # aprende a ESCONDER a origem
    def backward(ctx, g): return -lam*g
```

CORAL: mesmo "sotaque"

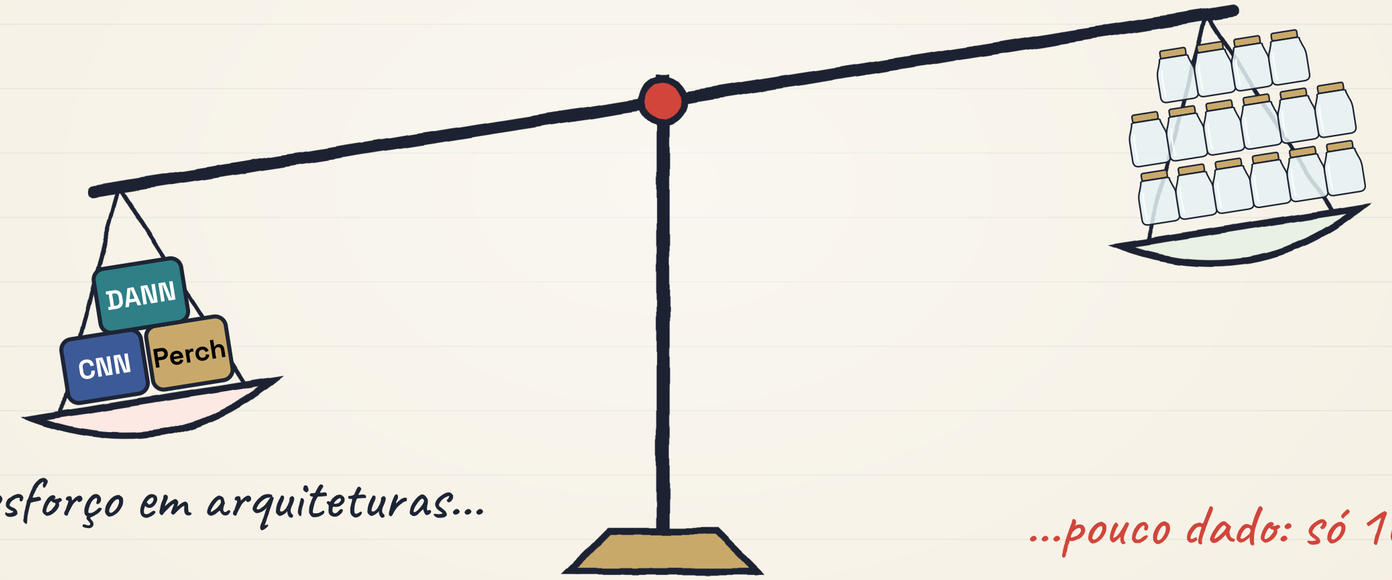
```python
# branqueia cada origem e recolore
# com a covariância do treino
Z = (Z - mean_o) @ inv_root_o @ root_alvo
```

Portão de concordância (v3): 0/197 Aedes e 0 alarmes falsos. Sem portão: 17/197 Aedes, com 20/371 outros mosquitos chamados de Aedes.

zero alarme falso ≠ detector bom

diagnóstico

O gargalo não é o modelo. São os dados.



muito esforço em arquiteturas...

...pouco dado: só 16 potes com Aedes

AUROC médio dos 8 candidatos entre **0,44 e 0,61**, com intervalos que se sobrepõem.

Fine-tuning na GPU (**0,584**) não superou o Perch congelado (**0,590**).

A origem segue decodificável: **94%** no Perch, até nos 9 descritores do batimento: **69%**.

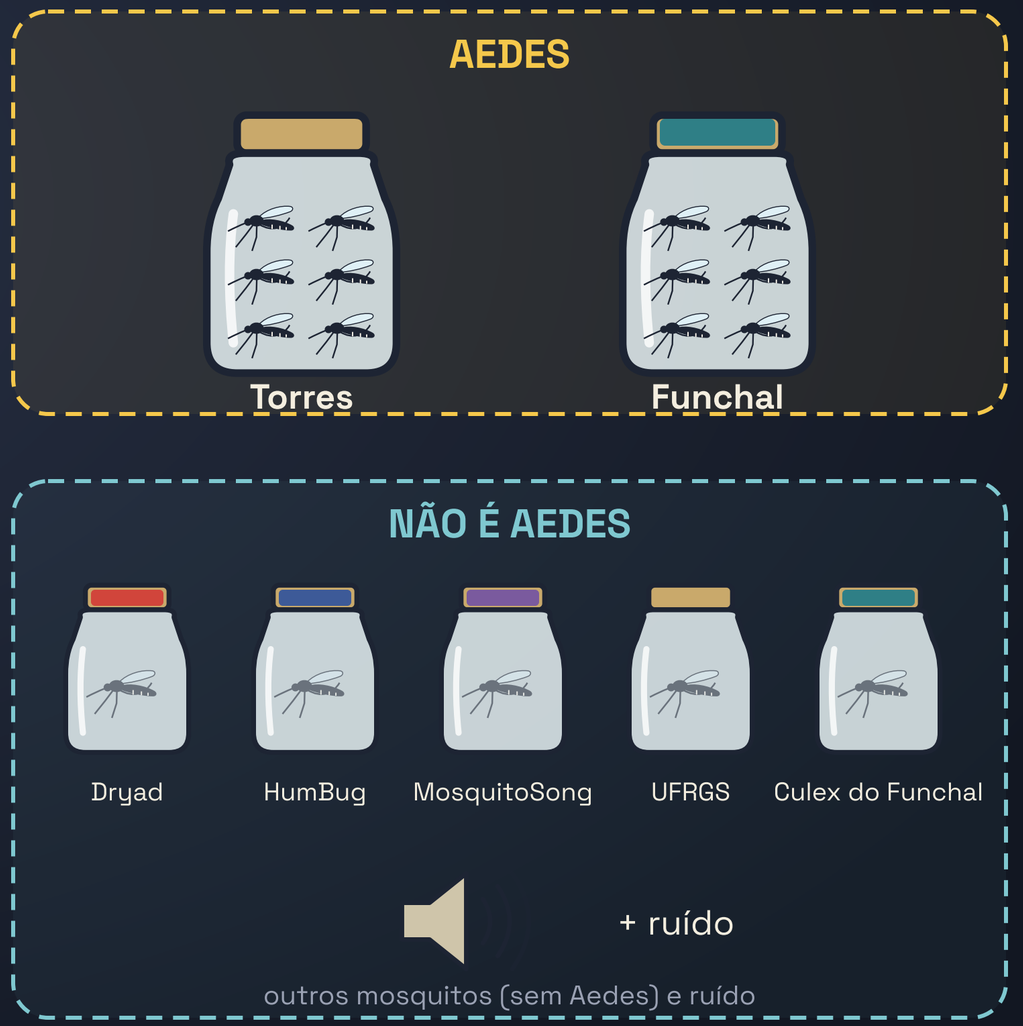
fonte: `reports/resultados_v4.md` · “Leitura”

parte 3 · a virada

Uma receita nova de dados

Se o problema é **de onde** vem cada Aedes, mudamos quem ensina o quê:

1. **Aedes** só de dois datasets grandes: **Torres** e **Funchal**.
2. Todos os outros datasets ensinam só o que **não é Aedes**: outros mosquitos e ruído.
3. A prova é sempre em **grupos e aparelhos fora do treino**.



como foi feito

Três ajustes no treino



cada aparelho tem seu "sotaque"

1 Normalizar por fonte

Cada fonte é centralizada nela mesma: some o "sotaque" do microfone, fica o mosquito.

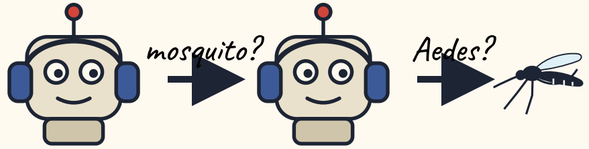
```
for fonte in fontes:
    m = origem == fonte
    X[m] = (X[m] - X[m].mean(0)) \
           / X[m].std(0)
```



2 Pesos por fonte

Uma fonte com 1.800 exemplos não pode "gritar" mais alto que uma com 20. Cada fonte pesa igual dentro da classe.

```
for fonte in fontes_da_classe:
    m = (y == c) & (origem == fonte)
    w[m] = 1 / m.sum() / n_fontes
modelo.fit(X, y, sample_weight=w)
```



3 Dois estágios

Primeiro: **tem mosquito?** Depois, só entre mosquitos: **é Aedes?** (média de dois modelos de árvores).

```
p_m = etapa1.predict_proba(X)[:, 1]
p_a = (hgb.predict_proba(X)[:, 1]
     + et.predict_proba(X)[:, 1]) / 2
p = [1 - p_m,           # ruído
     p_m * (1 - p_a),   # outro
     p_m * p_a]         # Aedes
```

resultado

Quando apita, costuma ser Aedes

0,83
AUROC para Aedes

79%
dos alertas de Aedes estão certos

8%
dos outros mosquitos viram alarme

1%
dos ruídos viram alarme

De cada 100 alertas de “Aedes”...



79 são Aedes de verdade 21 são outro mosquito ou ruído

e só 8% dos outros mosquitos (1% dos ruídos) disparam alarme

*Quando o modelo diz “é Aedes”, ele acerta **quase 8 em cada 10** vezes.*

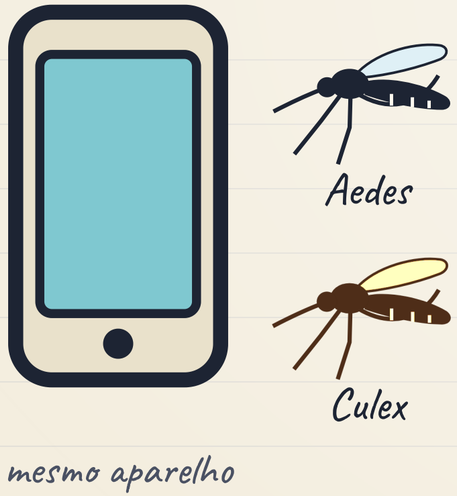
O modelo é **cauteloso**: prefere ficar quieto a dar alarme falso. Detecta **46%** dos Aedes, mas quando apita, costuma estar certo.

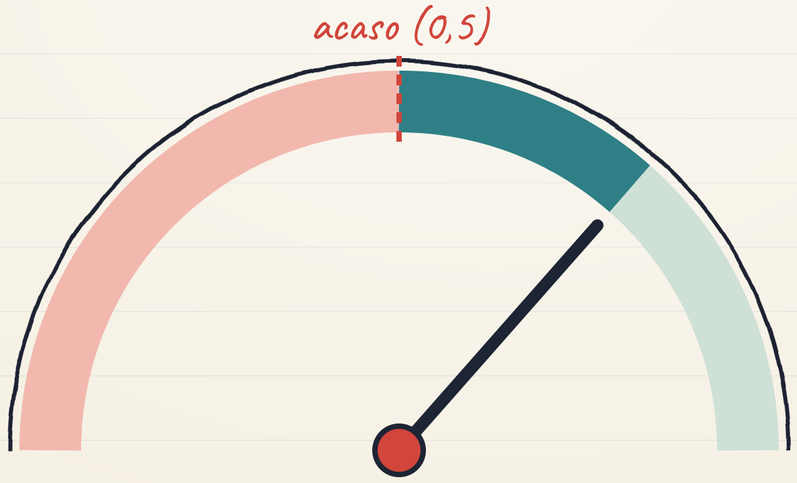
média de 3 repetições · validação cruzada por grupo

a prova mais justa

Aedes × *Culex* no mesmo aparelho

Aedes e *Culex* gravados nos **mesmos aparelhos**. Deixamos um aparelho inteiro fora do treino e perguntamos: o modelo separa as duas espécies?





0,73

AUROC Aedes × Culex · aparelho nunca visto

Por que essa prova vale mais

O aparelho é o mesmo para as duas espécies, então **o "sotaque" do microfone não ajuda**. Para acertar, o modelo precisa ouvir o mosquito.

Ainda falta

Os 95% do Torres não contam como prova: ele só tem Aedes. E Aedes gravados em **outros aparelhos** (Dryad, HumBug, MosquitoSong) ainda não são reconhecidos. O próximo passo é coletar.

o que falta para validar

Receita para um teste de verdade

- [ ] **Controles pareados:** Aedes *e* outras espécies gravados no **mesmo aparelho, setup e dia** — em vários aparelhos e locais.
- [ ] **Ruído in situ** dos ambientes reais, no lugar de ESC-50/Pete.
- [ ] **Normalizar o aparelho** (equalizar pelo fundo, descartar < 250 Hz), com emenda pré-registrada.
- [ ] **Um teste cego único**, numa campanha nova, com o protocolo congelado **antes** da coleta.



fonte: `reports/reanalise_v2.md §8` · `reports/datasets_candidatos_v4.md`

conclusão

O que aprendemos ouvindo mosquitos

1

Dentro das mesmas fontes há sinal (**AUROC 0,86–0,91**). Fora delas, o desempenho cai ao acaso.

2

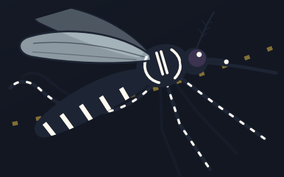
A **interpretabilidade** revelou o atalho: a origem é decodificável e as decisões dependem sobretudo de **130–280 Hz**.

3

Com a **receita certa de dados** (normalização por fonte, pesos e dois estágios): **79% de precisão** e Aedes × Culex no mesmo aparelho com AUROC **0,73**.

Obrigado!

perguntas? bzzz...



Luiz Eduardo do Nascimento Ciarallo · `mosquito-wingbeat`